PLC Based Four Floor Elevator Control System

Complete Ladder Logic Documentation

Shailesh Dhakal

Electrical Engineering

University of South Florida

Allen-Bradley Micro850 Simulator
Rockwell Automation Connected Components Workbench
Ladder Logic

I developed this four-floor elevator simulation to practice PLC sequencing, request handling, motion interlocks, fault diagnostics, and controlled recovery. This document explains the complete 77-rung program and includes the 22 selected ladder screenshots in program order.

The document records the implemented simulation logic. Functional test procedures and results are documented separately in the project report. The completed checklist contains 23 functional simulation checks, including the final travel watchdog retest and deliberate restart of a paused trip.

The supplied screenshots retain their original colors. The selected captures do not show a Windows taskbar clock or date. The superseded final capture is excluded and its replacement is used.

1 System Overview and State Definitions

The simulation represents floor positions with Sim_Position values of 0, 100, 200, and 300. Door travel is represented by Sim_DoorPosition, where 0 is closed and 100 is open. These are simulation units rather than physical measurements.

State	Operating step
0	Idle and destination selection
10	Upward travel
11	Downward travel
20	Door opening
30	Door dwell
40	Door closing
99	Latched fault

Normal service progresses from idle to travel, opening, dwell, closing, and idle. A request at the current floor enters the door cycle directly. An active destination stays locked until arrival; additional requests remain stored for later service.

The final timer settings are TON_SimTick = T#100ms, TON_Dwell = T#3s, and TON_Travel = T#60s. The travel watchdog was increased from 30 seconds after a valid long trip timed out prematurely, then normal operation and blocked travel were retested.

Normal Stop removes Run_Enable while preserving an active trip. Fault handling sets State to 99, clears requests, and disables operation. A successful reset returns to idle, and a fresh Start is required to enable operation.

2 How to Read the Ladder Logic

I organized the ladder as one routine, Prog1, under Program1 and MainTask. The following pages follow the displayed rung numbers. Some captures overlap so that a rung near a viewport edge can be read in the next image.

Contacts. A normally open contact passes logic when its Boolean tag is TRUE. A normally closed contact passes logic when its tag is FALSE. These symbols evaluate software values in this simulation.

Series and parallel paths. Conditions in series implement AND. Parallel branches implement OR. Each rung description states the conditions and the action at the right side.

Coils. A normal coil follows rung continuity. A latch coil marked L sets a bit until it is cleared. An unlatch coil marked U clears that bit.

Instructions. EQU checks equality; LES and GRT check less than and greater than. LEQ and GEQ include equality. MOV copies a value. ADD and SUB update the simulated position. TON is an on-delay timer with preset PT and completed output Q.

Scan order. The routine executes from top to bottom. Previous-input bits are updated after their edge-detection rungs. Destination candidates are written in a deliberate order, allowing the nearest eligible destination to overwrite a farther one before dispatch.

The screenshots were captured while CCW was disconnected. They document program structure and configured values rather than live execution. Abbreviated labels in the editor are written out in the explanations using the established project tag names.

This is a software simulation. The E-stop is evaluated by the scanned program; the implementation does not use an asynchronous interrupt or constitute a physical elevator safety system.

3 Simulation Fixtures and Lower Floor Feedback

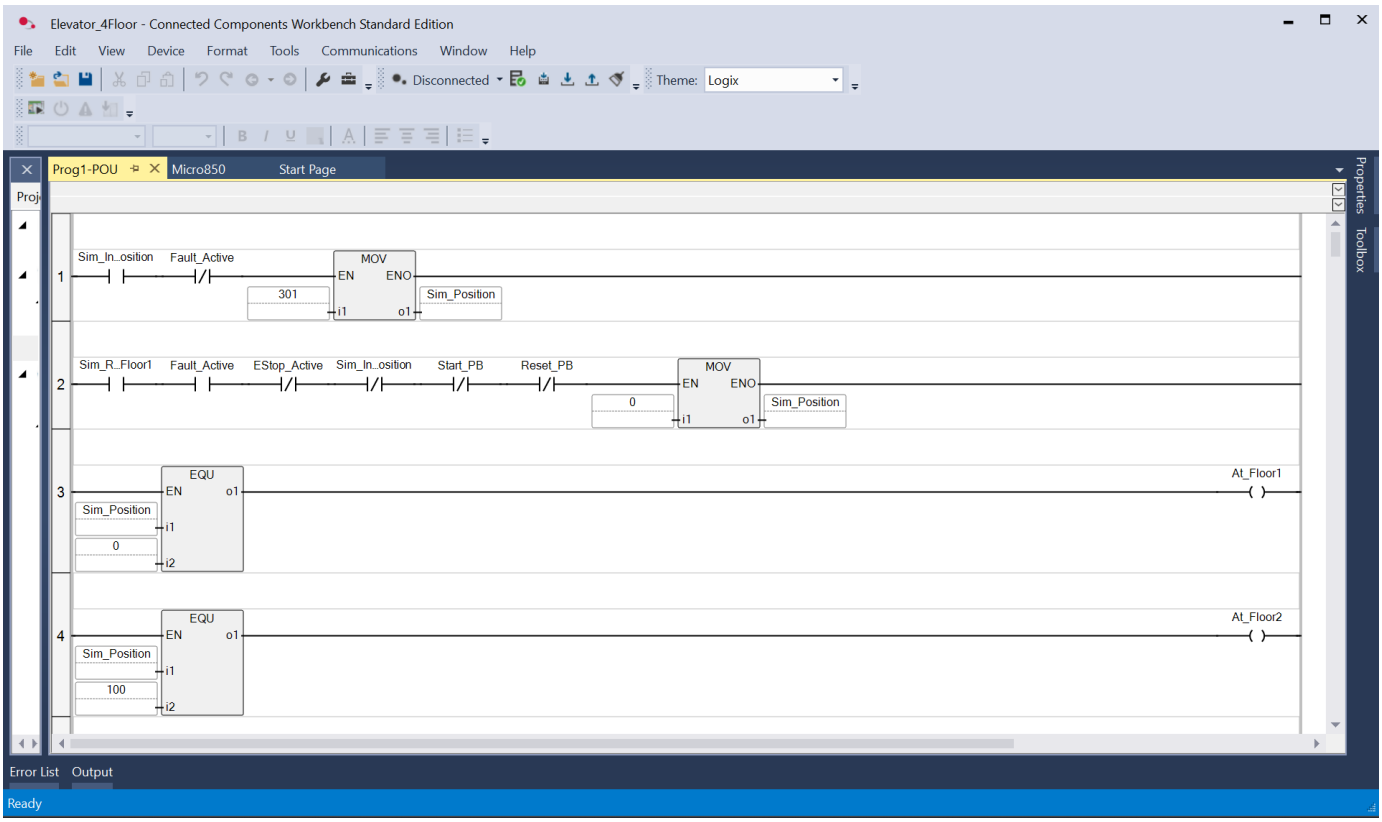


Figure 1. Ladder logic for simulation fixtures and lower floor feedback.

Rung 1. Sim_InvalidPosition TRUE and Fault_Active FALSE execute MOV 301 to Sim_Position. This creates an out-of-range position for fault injection.

Rung 2. Sim_ReturnFloor1 and Fault_Active must be TRUE, while EStop_Active, Sim_InvalidPosition, Start_PB, and Reset_PB must be FALSE. MOV then restores Sim_Position to 0 for controlled simulation recovery.

Rungs 3 and 4. EQU compares Sim_Position with 0 and 100. The normal coils At_Floor1 and At_Floor2 represent floor 1 and floor 2 feedback.

4 Upper Floor Feedback and Valid Floor Summary

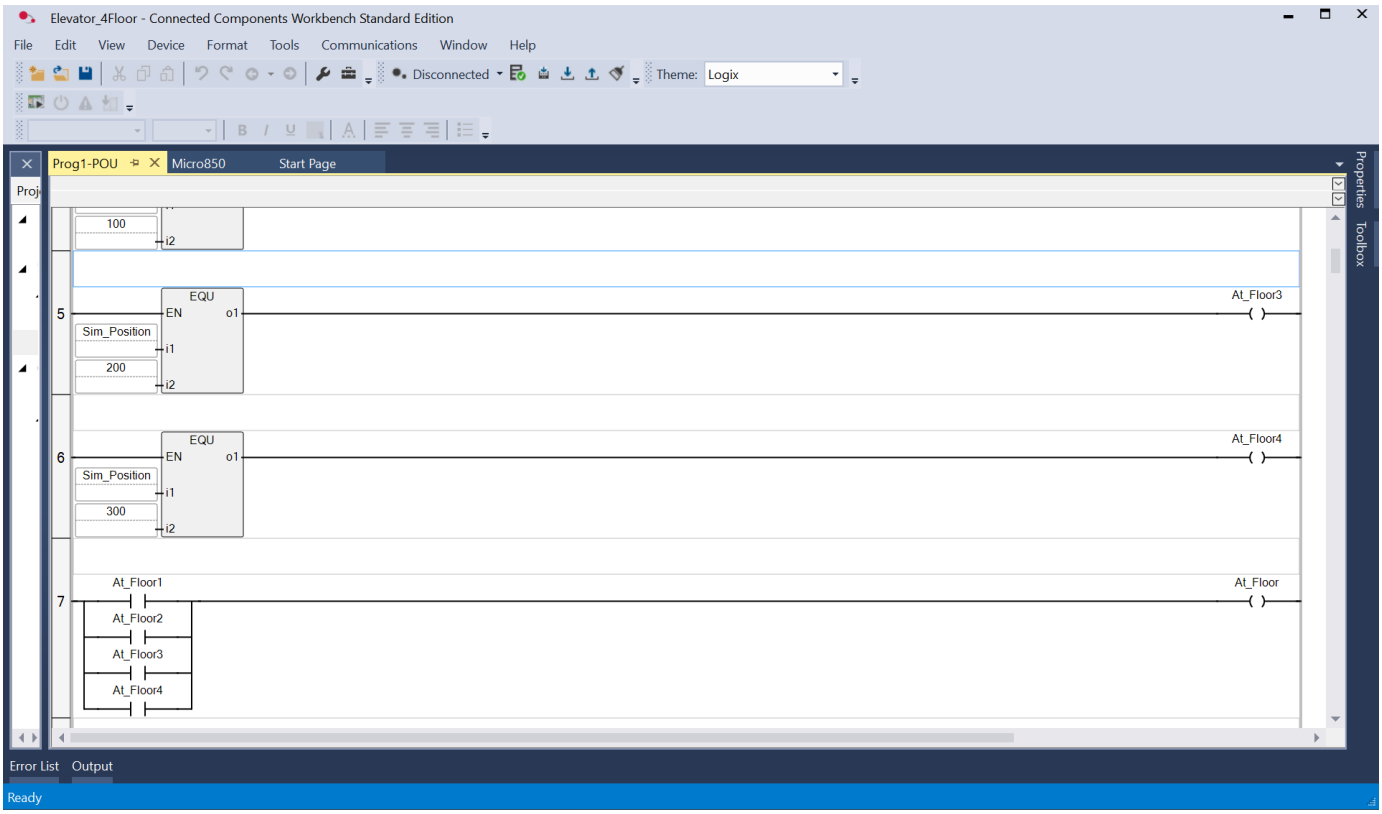


Figure 2. Ladder logic for upper floor feedback and valid floor summary.

Rung 5. Sim_Position equal to 200 energizes At_Floor3.

Rung 6. Sim_Position equal to 300 energizes At_Floor4.

Rung 7. At_Floor1 through At_Floor4 are parallel contacts. At_Floor is TRUE when any floor feedback bit is TRUE. Between floors, all four are FALSE and At_Floor is FALSE.

5 Current Floor Tracking

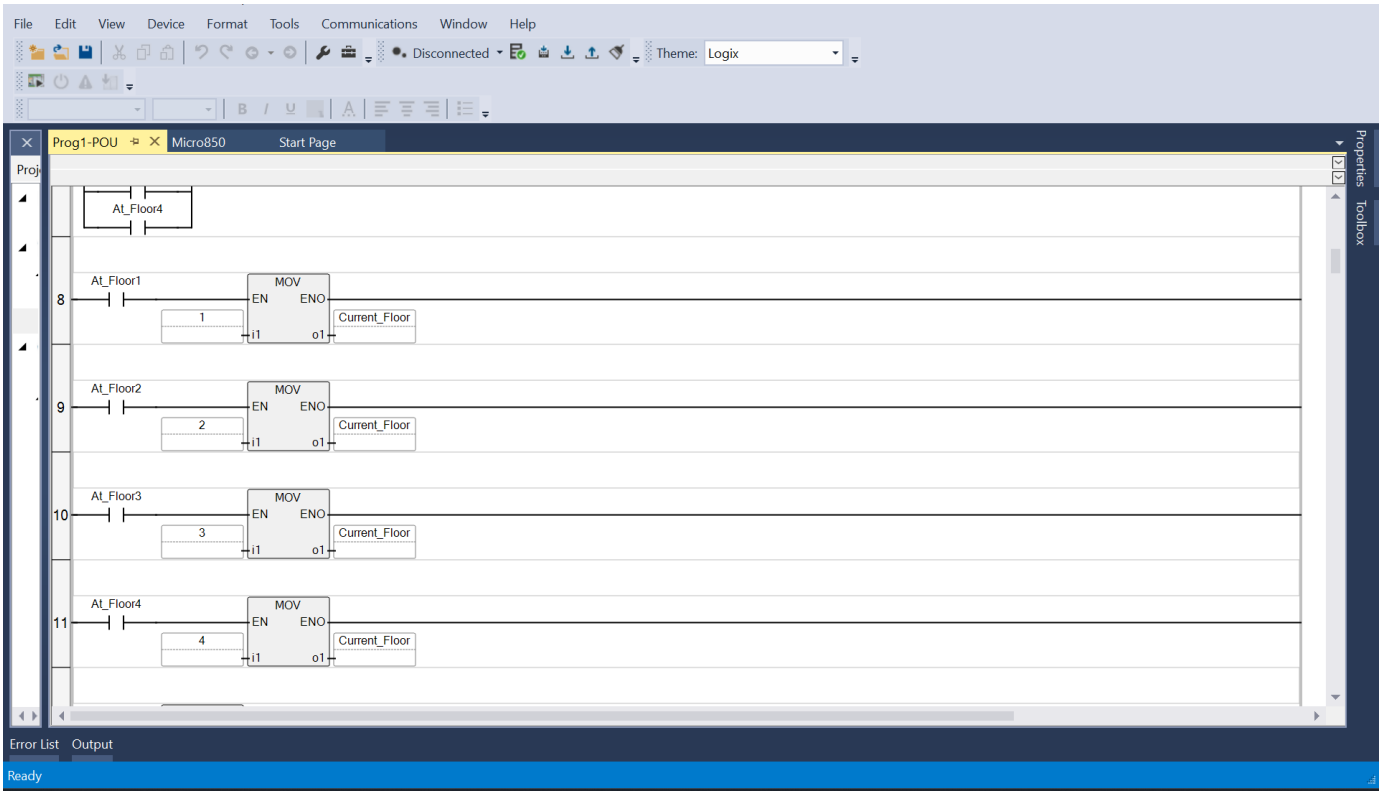


Figure 3. Ladder logic for current floor tracking.

Rung 8. At_Floor1 TRUE moves 1 into Current_Floor.

Rung 9. At_Floor2 TRUE moves 2 into Current_Floor.

Rung 10. At_Floor3 TRUE moves 3 into Current_Floor.

Rung 11. At_Floor4 TRUE moves 4 into Current_Floor.

Tracking behavior. These MOV instructions execute only at their mapped positions. Between floors, Current_Floor retains the last reached floor; it is not an indication that the car is currently aligned with that floor.

6 Door Feedback and Simulation Tick

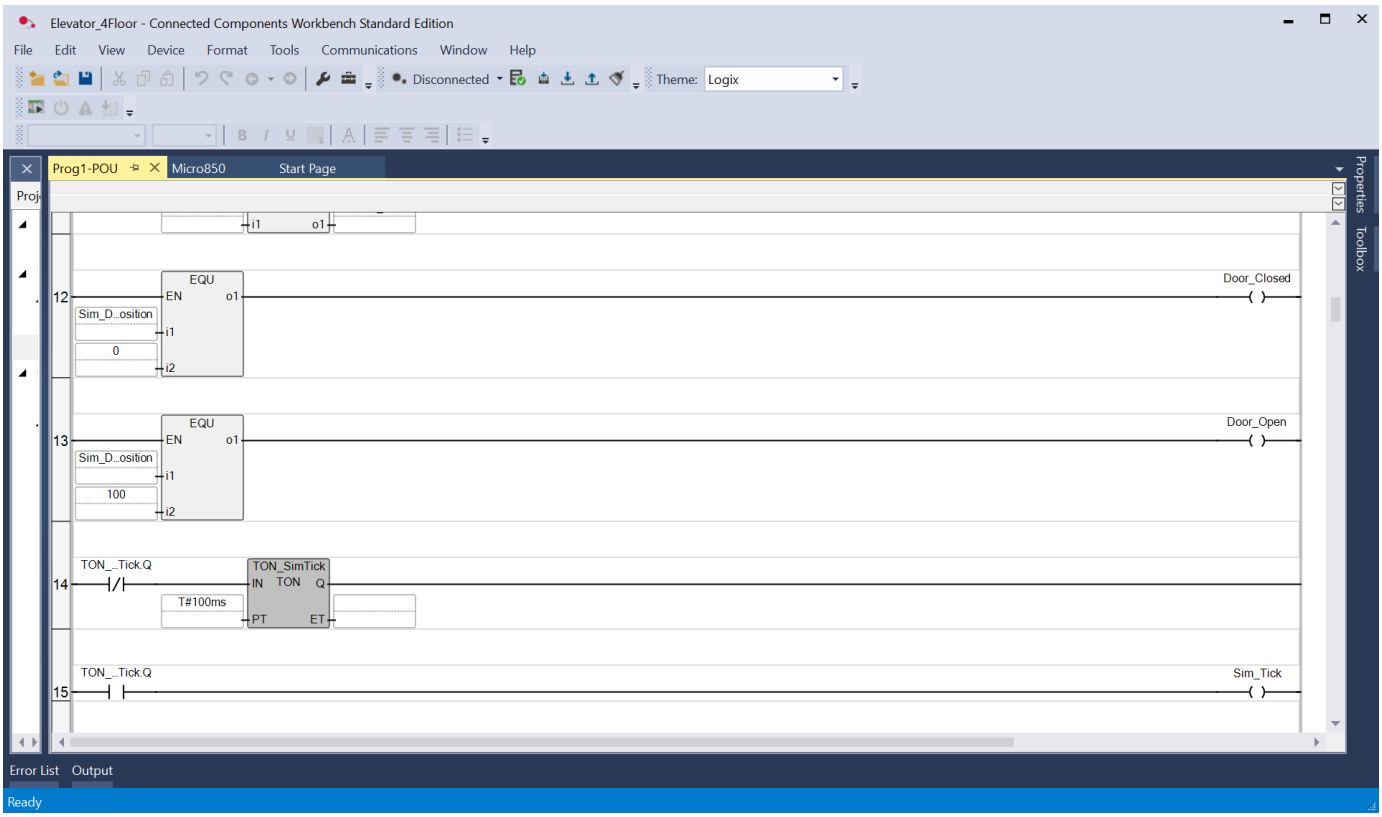


Figure 4. Ladder logic for door feedback and simulation tick.

Rung 12. Sim_DoorPosition equal to 0 energizes Door_Closed.

Rung 13. Sim_DoorPosition equal to 100 energizes Door_Open.

Rung 14. The normally closed TON_SimTick.Q contact enables TON_SimTick with PT = T#100ms. When Q becomes TRUE, the next scan disables and resets the timer, allowing the cycle to repeat.

Rung 15. TON_SimTick.Q drives Sim_Tick. This pulse is used by the position-update rungs. The nominal tick interval includes scan effects and should not be treated as an independently measured exact 100 ms period.

7 Position Validation

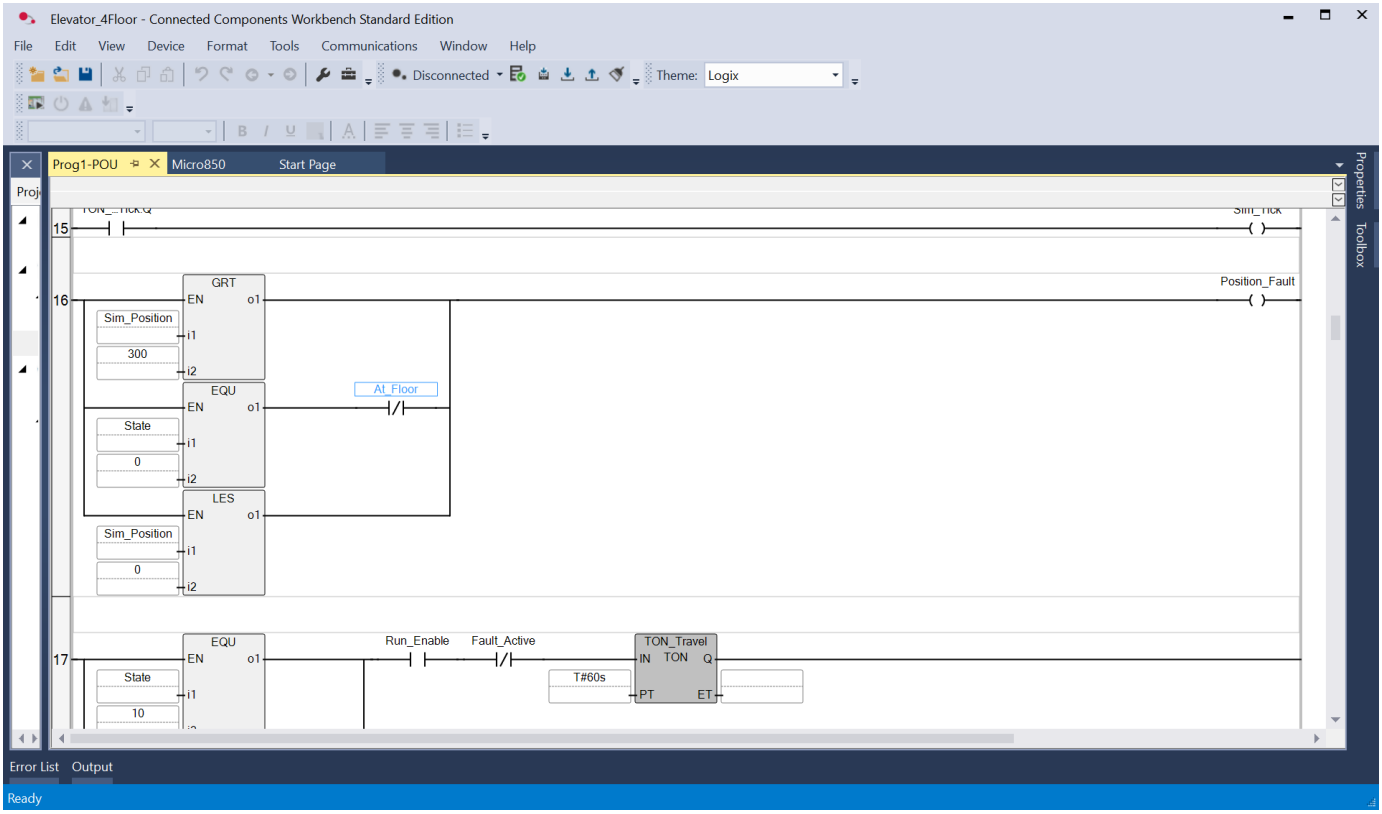


Figure 5. Ladder logic for position validation.

Rung 16. Three parallel paths energize Position_Fault: Sim_Position greater than 300; State equal to 0 with At_Floor FALSE; or Sim_Position less than 0.

Idle validity. An intermediate position is permitted during travel. The State = 0 branch specifically detects an idle condition without a recognized floor.

Next rung. The lower part of the capture begins the travel watchdog. Its complete parallel state conditions appear on the following page.

8 Travel Watchdog and Emergency Stop Code

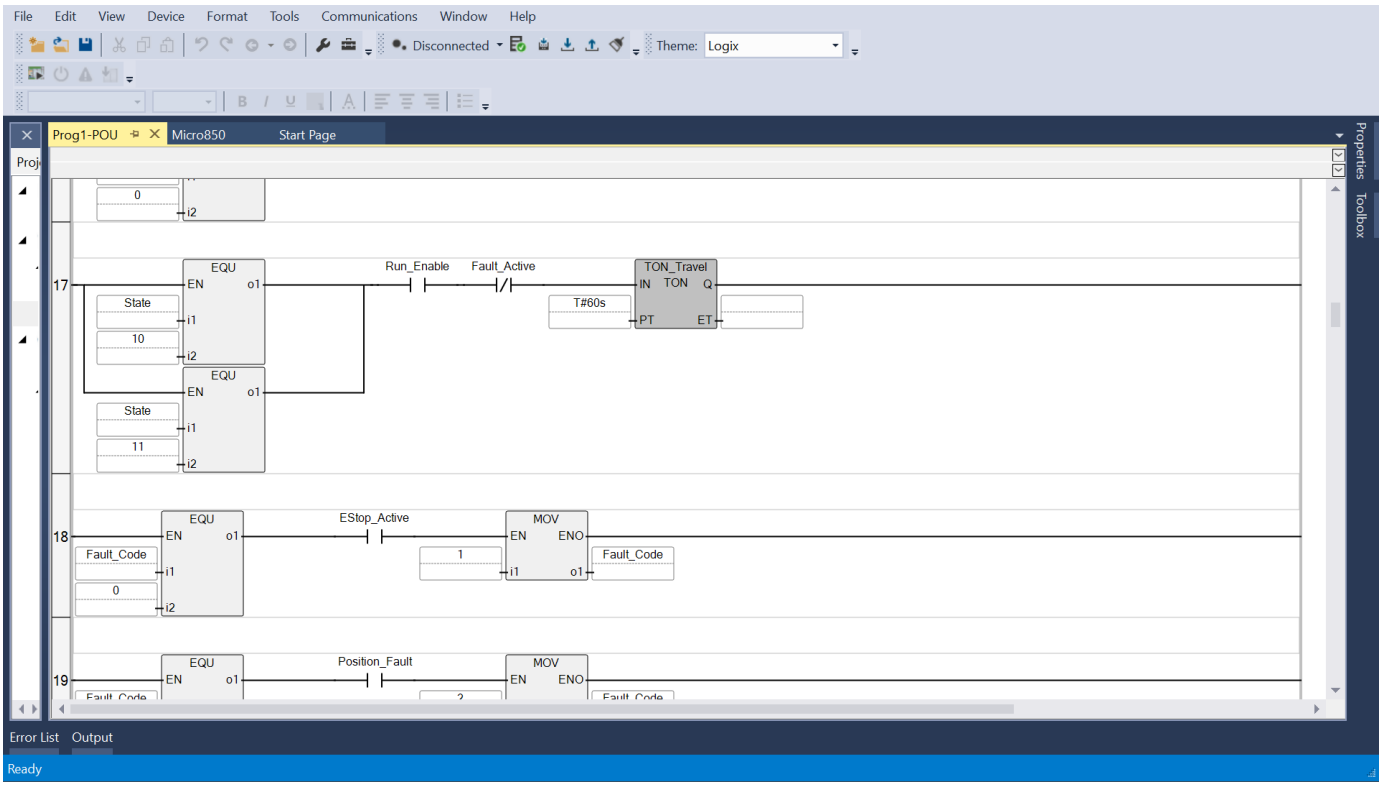


Figure 6. Ladder logic for travel watchdog and emergency stop code.

Rung 17. State = 10 and State = 11 form parallel paths. Their result passes through Run_Enable TRUE and Fault_Active FALSE to TON_Travel. PT is T#60s. Continuous enabled travel beyond this preset sets Q.

Rung 18. Fault_Code = 0 and EStop_Active TRUE move 1 to Fault_Code. The zero-code condition prevents overwriting an already recorded fault.

Watchdog behavior. Normal Stop removes Run_Enable and resets this nonretentive timer. The watchdog therefore measures enabled travel rather than accumulated time across pauses.

9 Fault Code Capture and Fault State

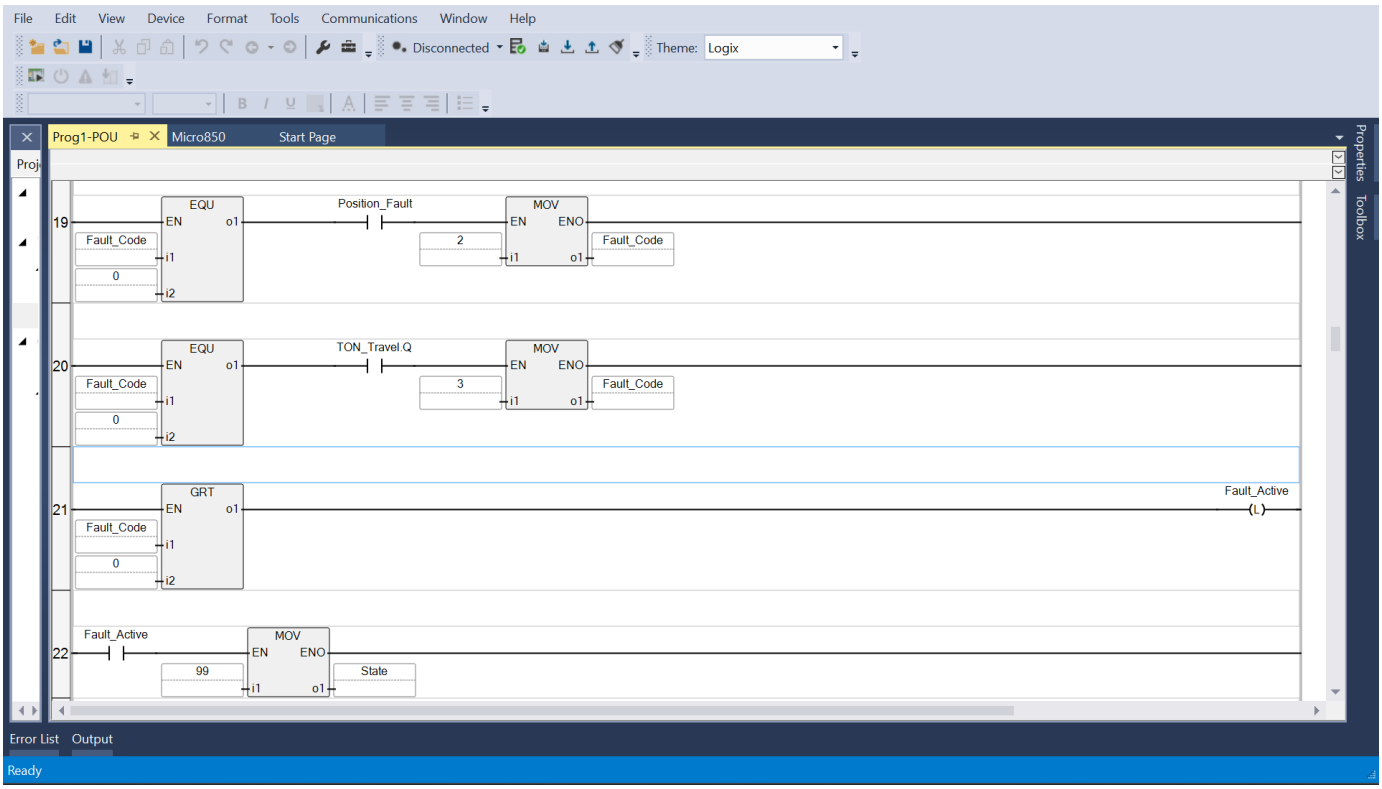


Figure 7. Ladder logic for fault code capture and fault state.

Rung 19. Fault_Code = 0 and Position_Fault TRUE move 2 to Fault_Code.

Rung 20. Fault_Code = 0 and TON_Travel.Q TRUE move 3 to Fault_Code.

Rung 21. Fault_Code greater than 0 latches Fault_Active.

Rung 22. Fault_Active TRUE moves 99 to State. Fault detection is above normal sequence handling. If several new causes occur in the same scan with code 0, rung order records E-stop before position fault before travel timeout.

10 Clearing Requests During a Fault

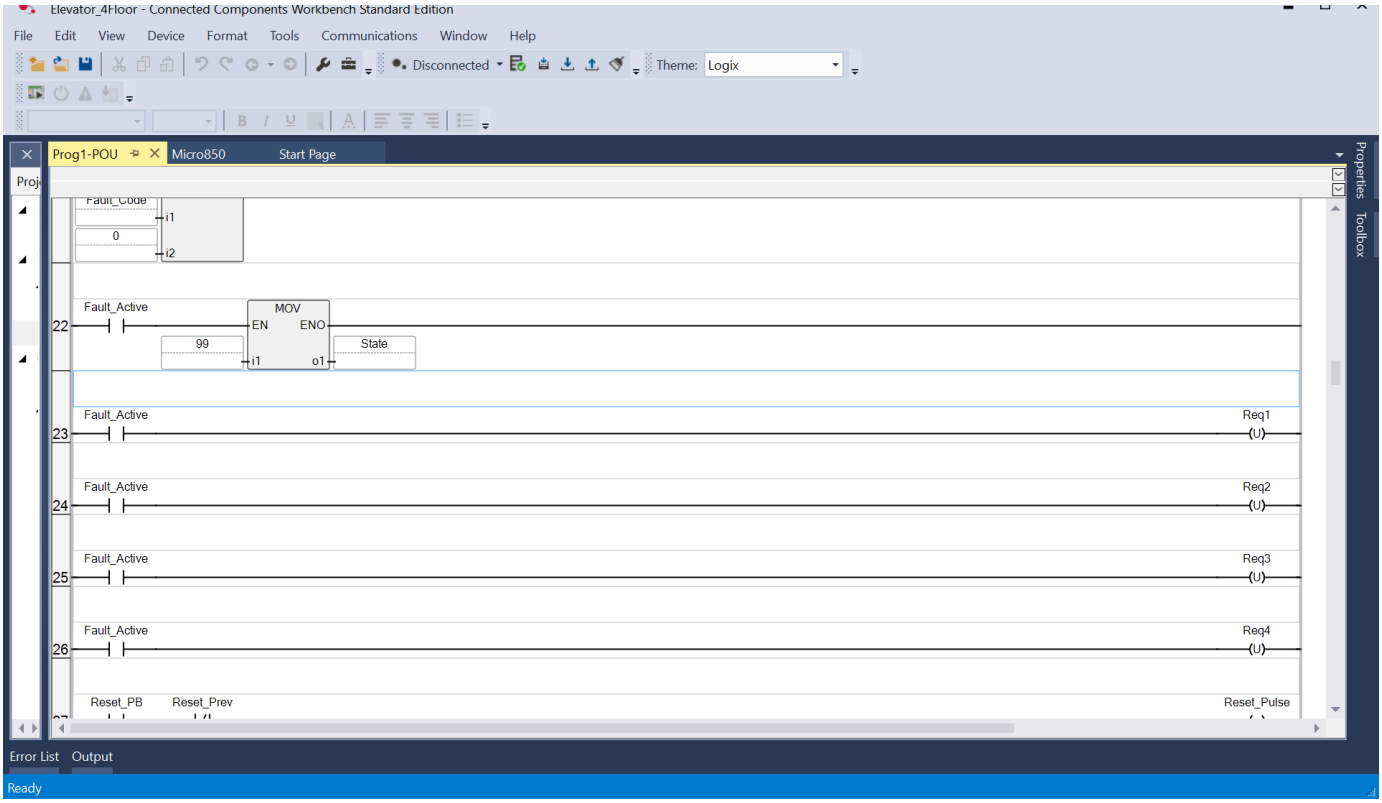


Figure 8. Ladder logic for clearing requests during a fault.

Rung 22. The repeated rung shows the fault override to State = 99.

Rungs 23 to 26. Each rung uses Fault_Active to unlatch one stored request: Req1, Req2, Req3, and Req4 respectively.

Effect. A fault discards the pending service requests. This differs from normal Stop, which retains the active request and destination for deliberate resumption. These rungs do not by themselves clear the numeric Target_Position; idle handling updates it after recovery.

11 Reset Detection and Permission

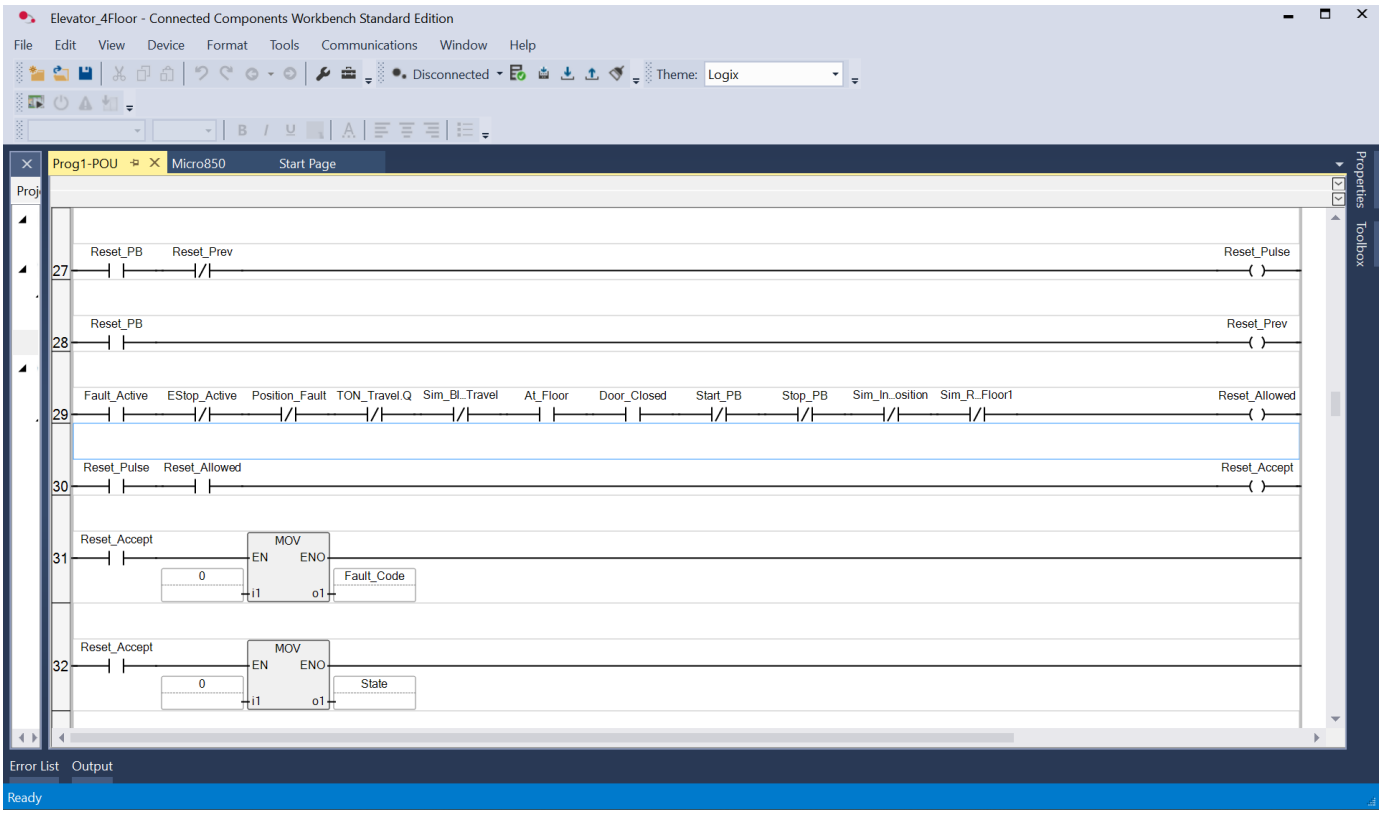


Figure 9. Ladder logic for reset detection and permission.

Rung 27. Reset_PB TRUE with Reset_Prev FALSE generates Reset_Pulse for the rising edge.

Rung 28. Reset_PB updates Reset_Prev after the pulse evaluation.

Rung 29. Reset_Allowed requires Fault_Active, At_Floor, and Door_Closed TRUE. EStop_Active, Position_Fault, TON_Travel.Q, Sim_BlockTravel, Start_PB, Stop_PB, Sim_InvalidPosition, and Sim_ReturnFloor1 must be FALSE.

Rungs 30 to 32. Reset_Pulse AND Reset_Allowed produce Reset_Accept. That signal moves 0 to Fault_Code and 0 to State. Fault unlatching follows on rung 33.

12 Accepted Reset Start Edge and Run Enable

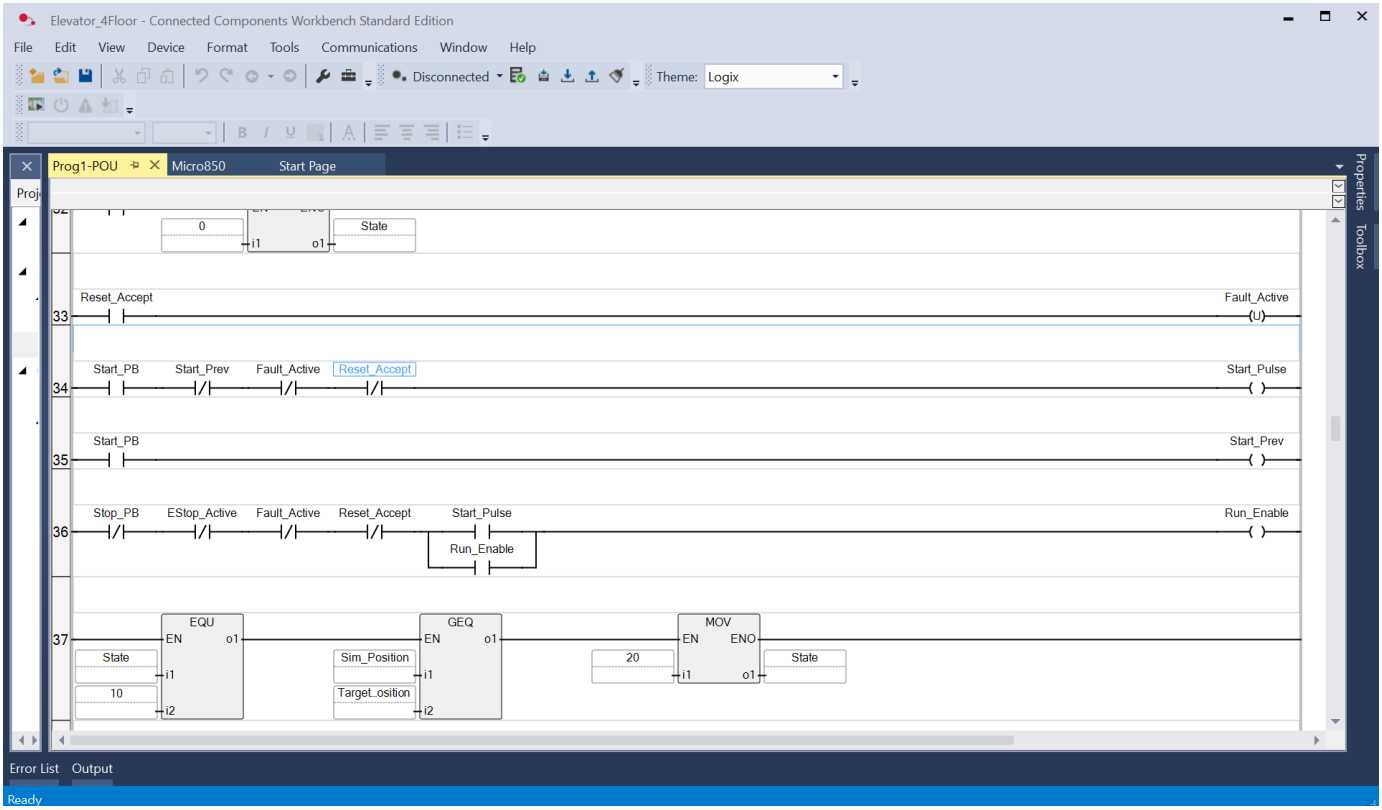


Figure 10. Ladder logic for accepted reset start edge and run enable.

Rung 33. Reset_Accept unlatches Fault_Active.

Rungs 34 and 35. Start_PB TRUE and Start_Prev FALSE create Start_Pulse only when Fault_Active and Reset_Accept are FALSE. Start_Prev is then updated from Start_PB.

Rung 36. Stop_PB, EStop_Active, Fault_Active, and Reset_Accept must all be FALSE. A parallel Start_Pulse or Run_Enable contact then drives Run_Enable, creating seal-in and removing enable on Stop or fault.

Rung 37. State = 10 and Sim_Position greater than or equal to Target_Position move 20 to State, initiating arrival door service.

13 Downward Arrival and Travel Commands

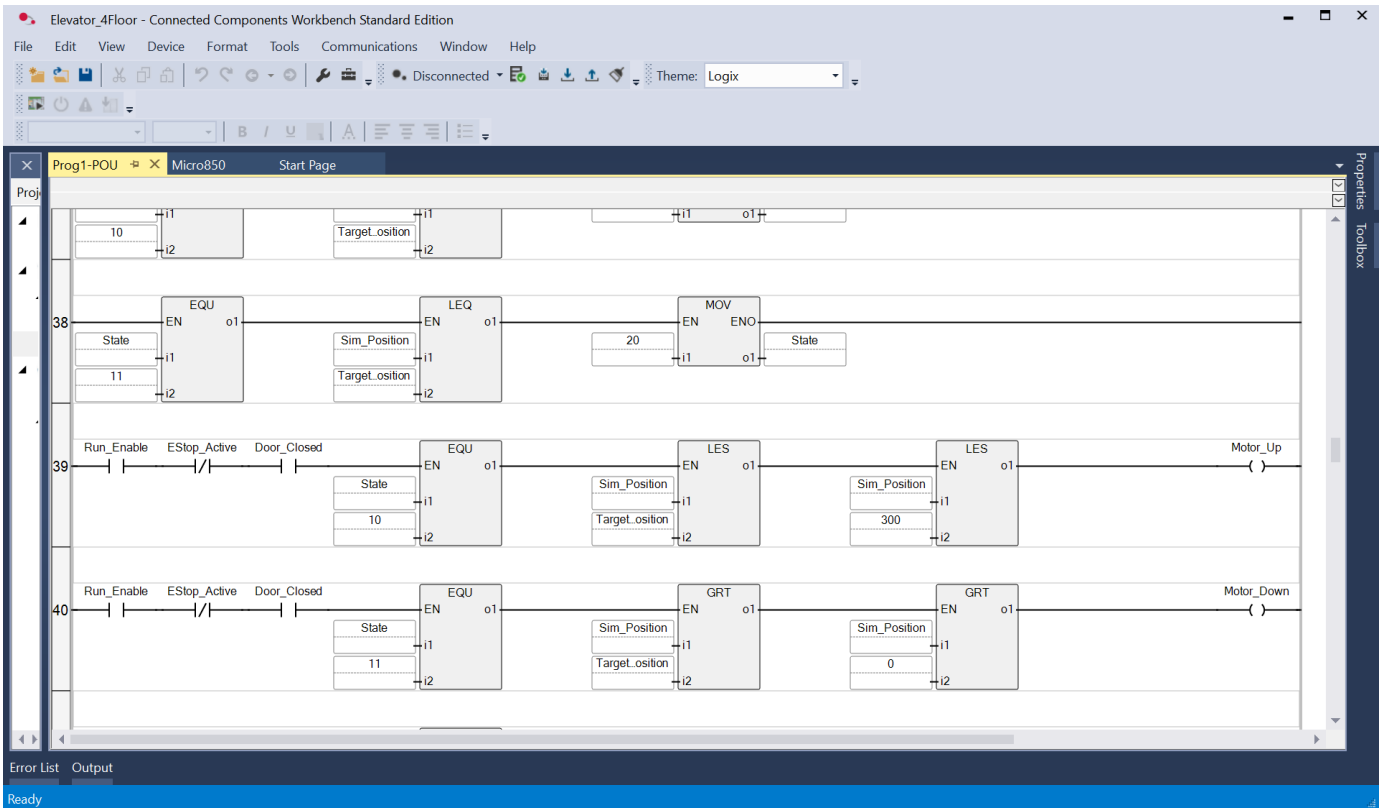


Figure 11. Ladder logic for downward arrival and travel commands.

Rung 38. State = 11 and Sim_Position less than or equal to Target_Position move 20 to State.

Rung 39. Run_Enable TRUE, EStop_Active FALSE, Door_Closed TRUE, State = 10, Sim_Position below Target_Position, and Sim_Position below 300 energize Motor_Up.

Rung 40. The corresponding downward conditions require State = 11, Sim_Position above Target_Position, and Sim_Position above 0 to energize Motor_Down.

Interlock. The single State value selects one travel direction. Door_Closed is required in both command rungs. Arrival changes the state before these commands are evaluated.

14 Simulated Travel and Open Door Transition

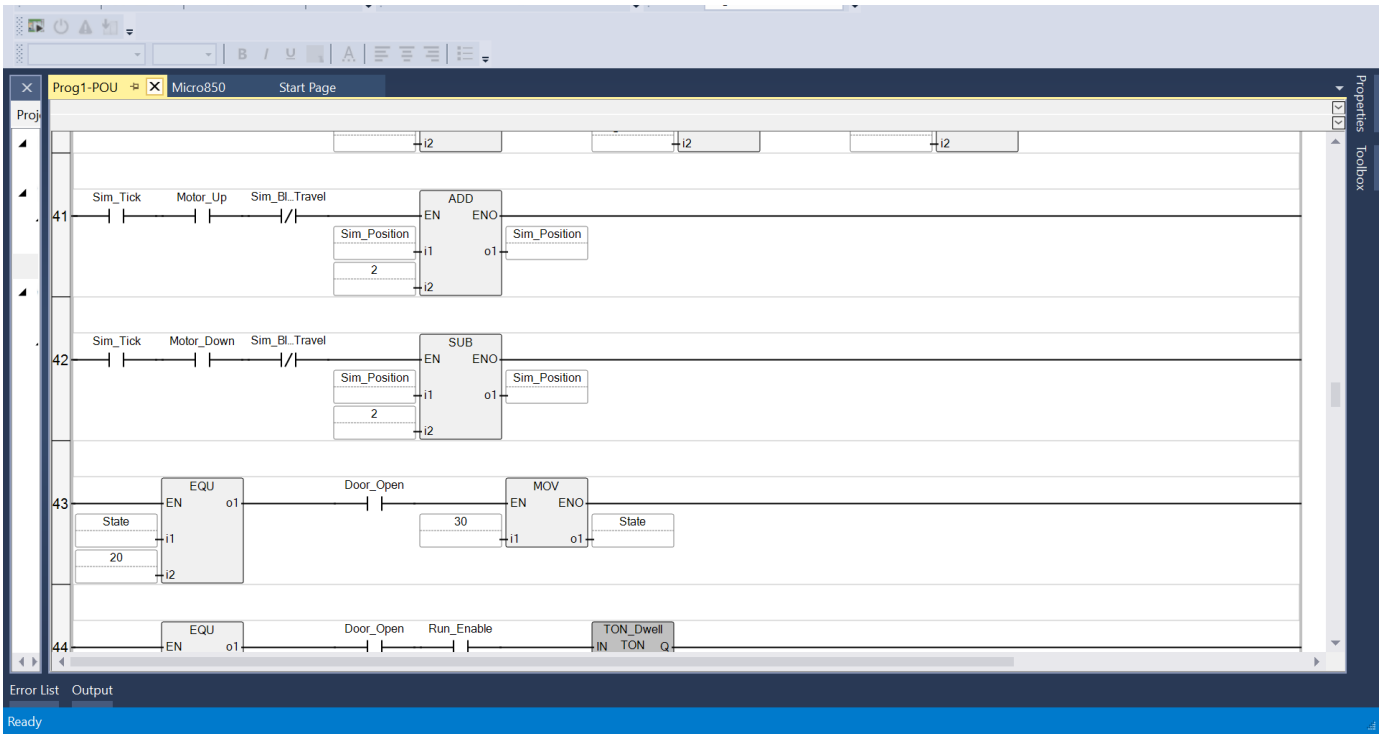


Figure 12. Ladder logic for simulated travel and open door transition.

Rung 41. Sim_Tick and Motor_Up TRUE with Sim_BlockTravel FALSE add 2 to Sim_Position.

Rung 42. Sim_Tick and Motor_Down TRUE with Sim_BlockTravel FALSE subtract 2 from Sim_Position.

Rung 43. State = 20 and Door_Open TRUE move 30 to State.

Fault injection. Sim_BlockTravel prevents these numeric position updates while a travel state remains active. This supports a repeatable watchdog test without changing the main destination logic.

15 Door Dwell and Closing Transition

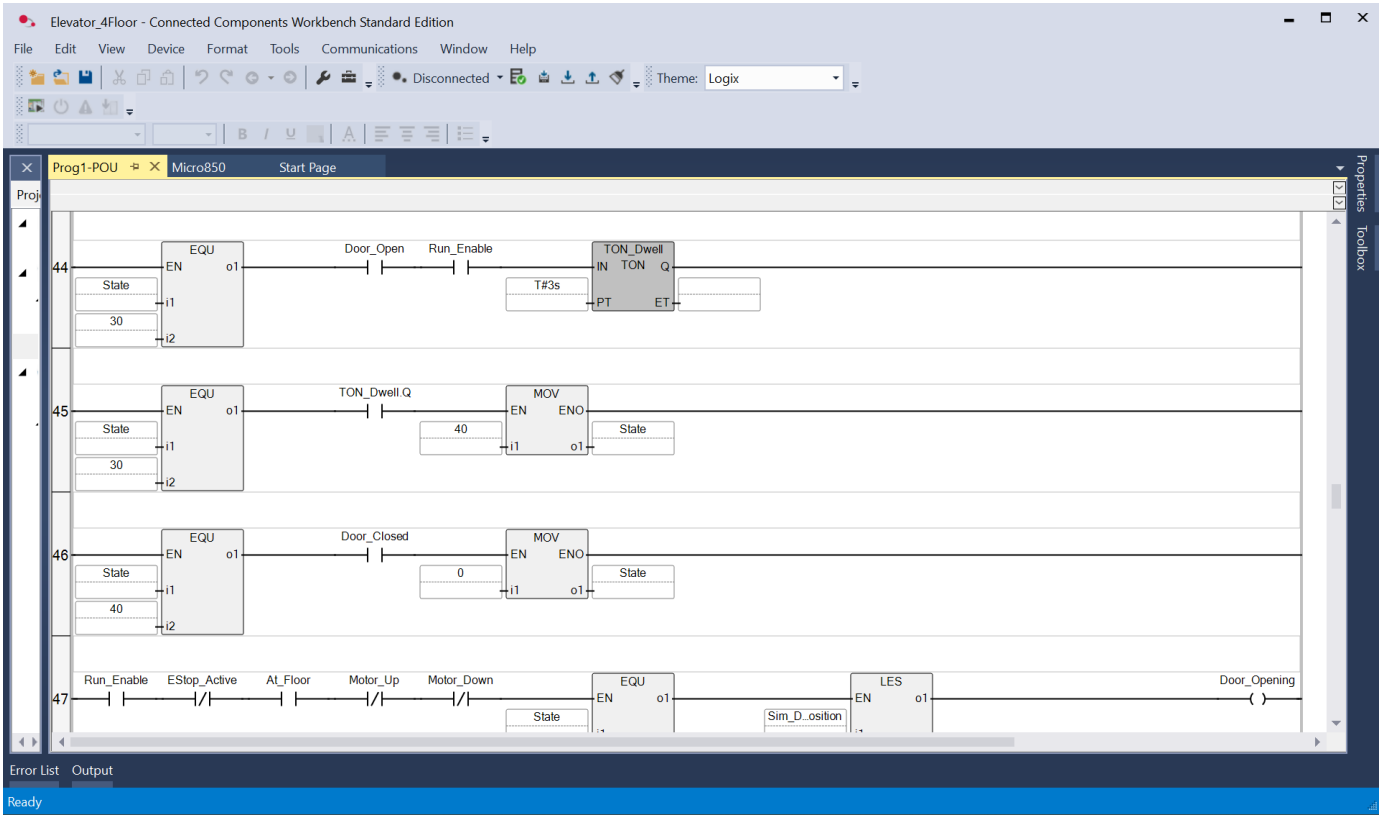


Figure 13. Ladder logic for door dwell and closing transition.

Rung 44. State = 30, Door_Open TRUE, and Run_Enable TRUE enable TON_Dwell with PT = T#3s.

Rung 45. State = 30 and TON_Dwell.Q TRUE move 40 to State, beginning closing.

Rung 46. State = 40 and Door_Closed TRUE move 0 to State, completing the service cycle.

Timer behavior. The dwell timer is nonretentive. Removing its enabling conditions resets it. Opening and closing command details appear on the next page.

16 Door Motion Interlocks and Opening Update

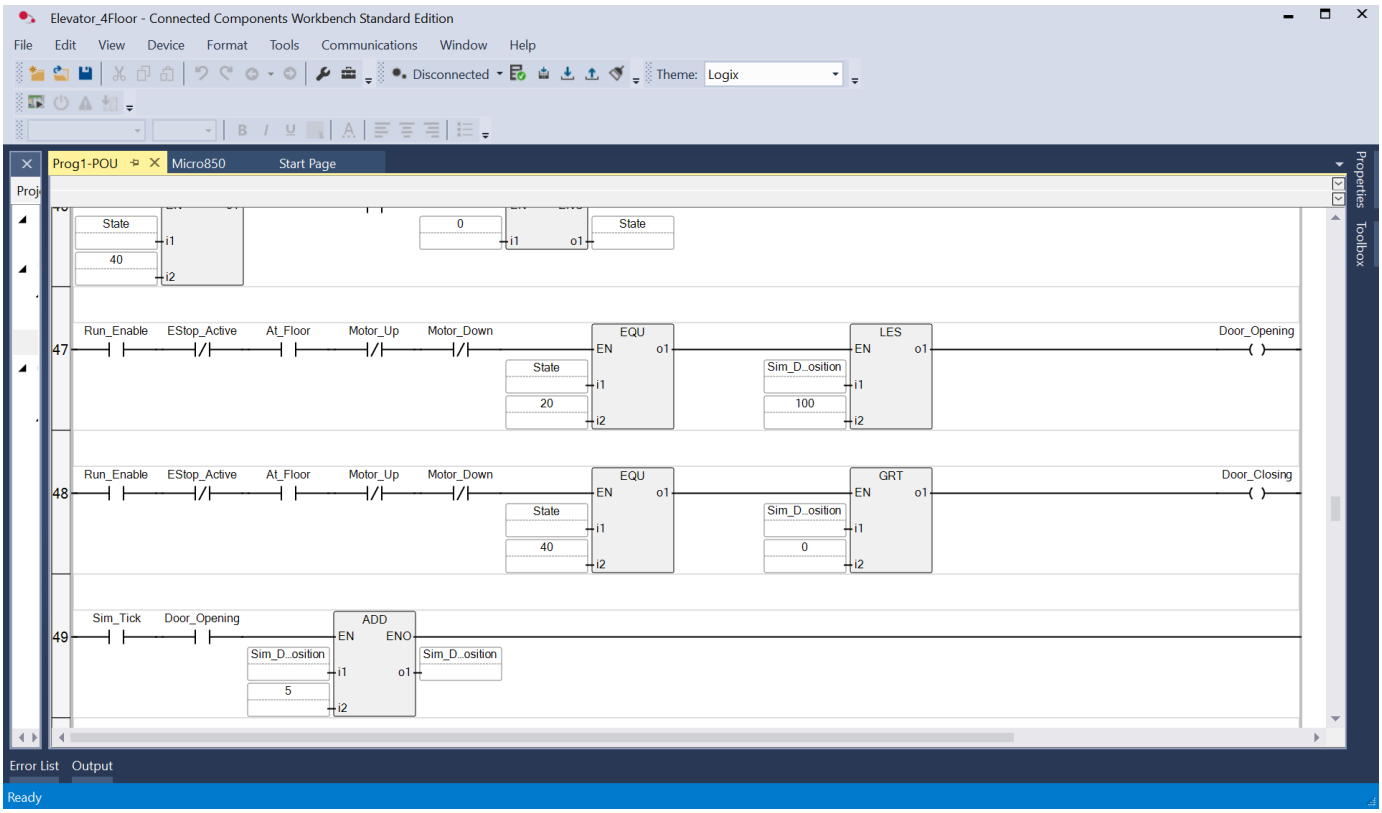


Figure 14. Ladder logic for door motion interlocks and opening update.

Rung 47. Run_Enable and At_Floor must be TRUE; EStop_Active, Motor_Up, and Motor_Down must be FALSE. State = 20 and Sim_DoorPosition below 100 then energize Door_Opening.

Rung 48. The same common permissives with State = 40 and Sim_DoorPosition above 0 energize Door_Closing.

Rung 49. Sim_Tick and Door_Opening TRUE add 5 to Sim_DoorPosition. Door movement is permitted only at a recognized floor with both motor commands OFF.

17 Closing Update and Request Capture

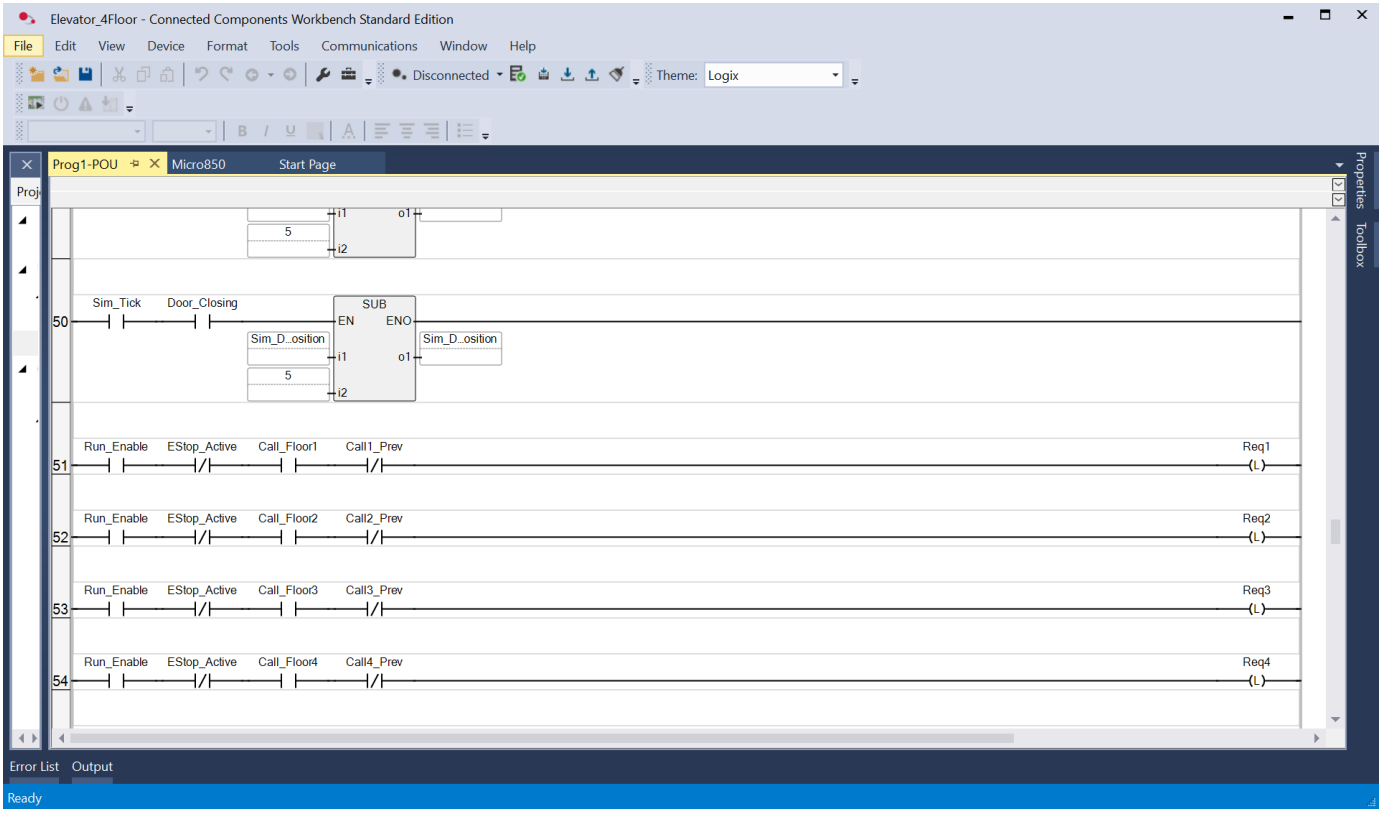


Figure 15. Ladder logic for closing update and request capture.

Rung 50. Sim_Tick and Door_Closing TRUE subtract 5 from Sim_DoorPosition.

Rungs 51 to 54. Each floor has an independent request latch. Run_Enable TRUE, EStop_Active FALSE, Call_FloorN TRUE, and CallN_Prev FALSE latch ReqN for floors 1 through 4.

Stored request behavior. Releasing a call does not unlatch its request. Holding a call TRUE does not repeatedly create new rising edges. Calls entered while disabled are not stored by these rungs.

18 Previous Call Values and Floor One Clearing

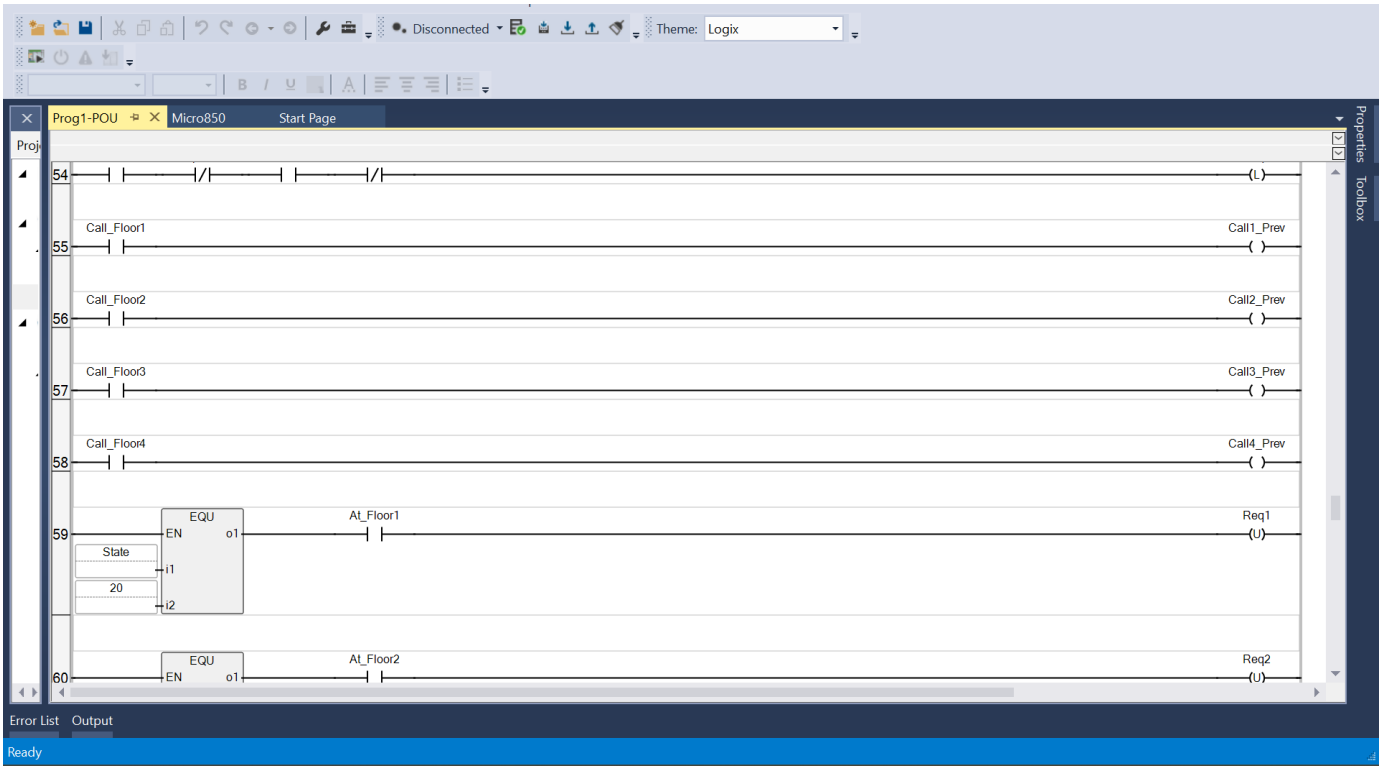


Figure 16. Ladder logic for previous call values and floor one clearing.

Rungs 55 to 58. Call_Floor1 through Call_Floor4 update Call1_Prev through Call4_Prev. These normal coils execute after the request capture rungs so each rising edge is detected using the previous scan value.

Rung 59. State = 20 and At_Floor1 TRUE unlatch Req1.

Service timing. The request clears during the opening state at its destination. It does not wait for the entire opening, dwell, and closing sequence to finish. The start of floor 2 clearing is repeated in the next capture.

19 Clearing Served Requests at Floors Two to Four

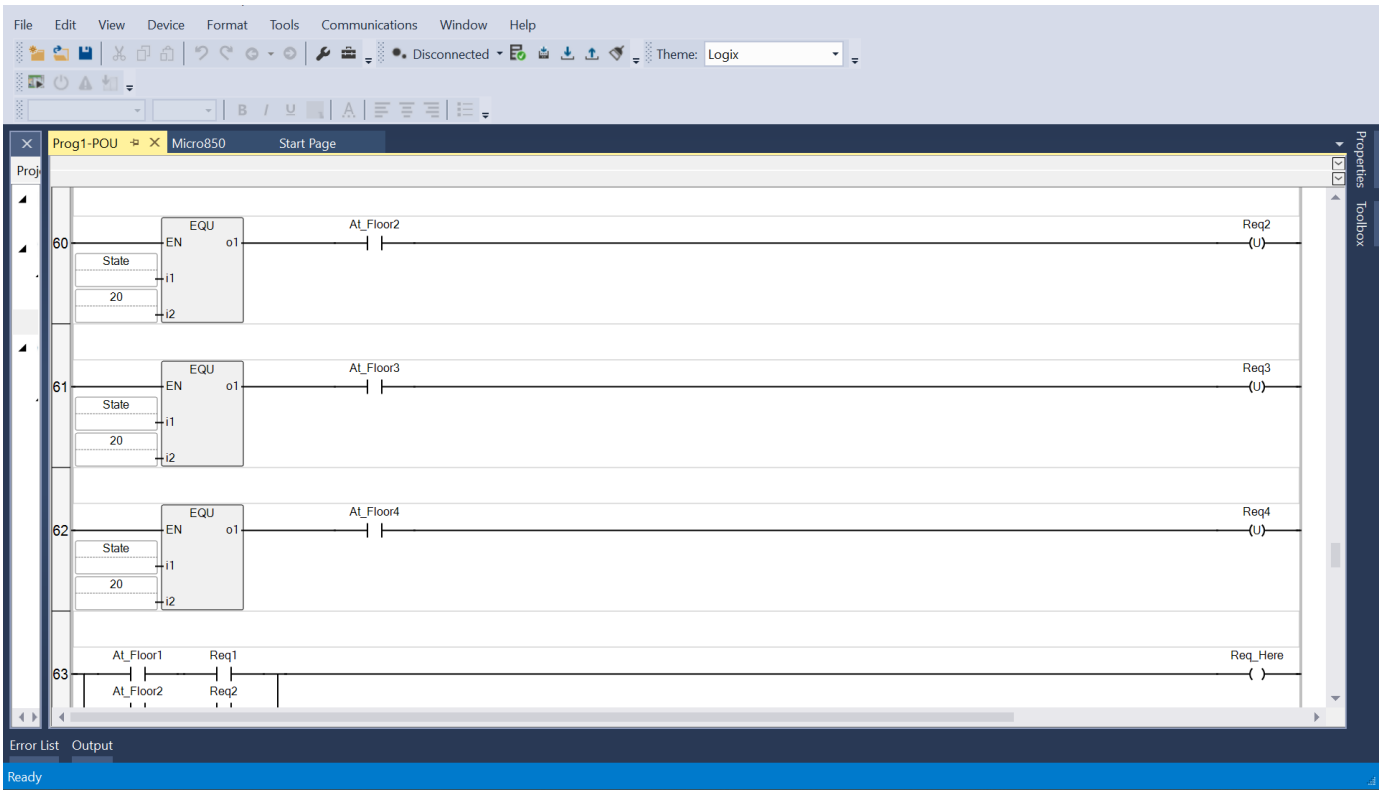


Figure 17. Ladder logic for clearing served requests at floors two to four.

Rung 60. State = 20 and At_Floor2 TRUE unlatch Req2.

Rung 61. State = 20 and At_Floor3 TRUE unlatch Req3.

Rung 62. State = 20 and At_Floor4 TRUE unlatch Req4.

Request summary. The next rung begins the Req_Here summary. Its complete four-branch arrangement is shown on the following page.

20 Requests Here and Above

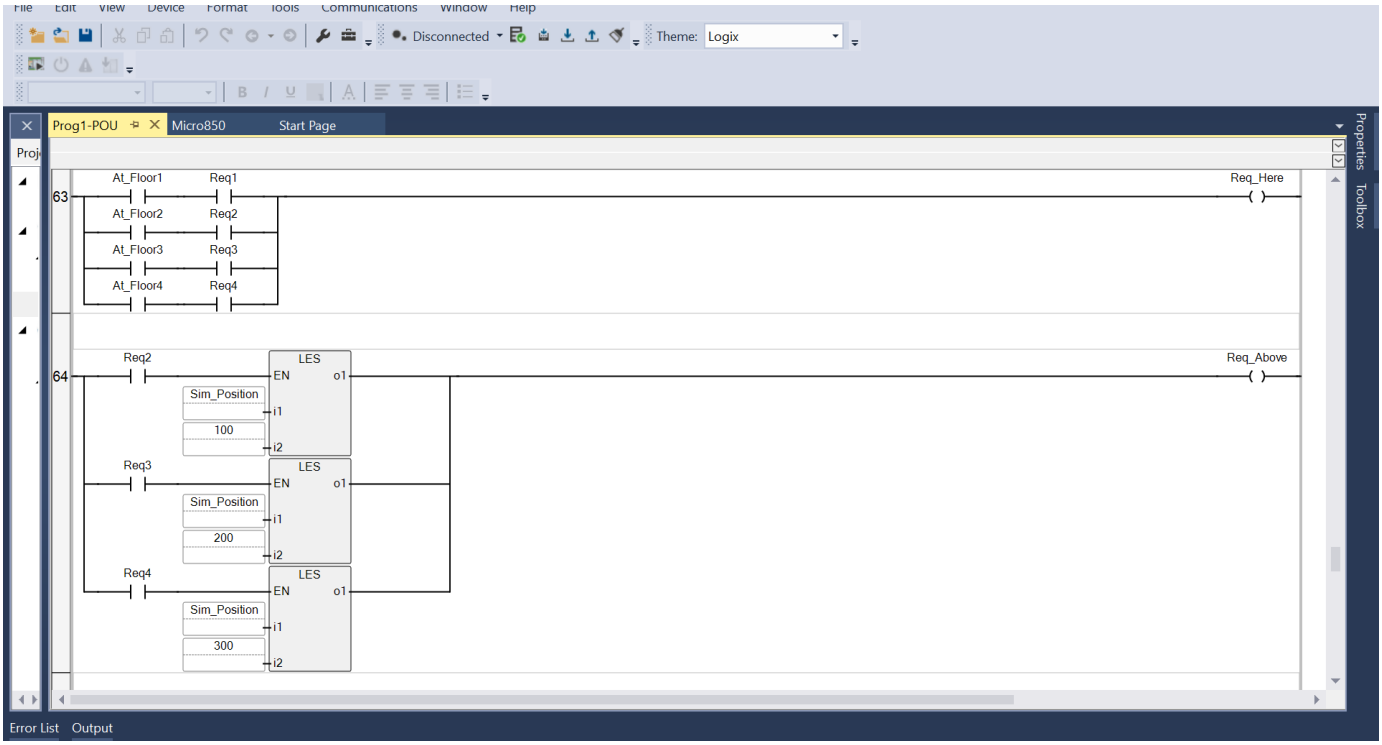


Figure 18. Ladder logic for requests here and above.

Rung 63. Four parallel branches pair At_Floor1 with Req1, At_Floor2 with Req2, At_Floor3 with Req3, and At_Floor4 with Req4. Any true pair energizes Req_Here.

Rung 64. Req_Above is TRUE for Req2 with position below 100, Req3 with position below 200, or Req4 with position below 300. The three paths are parallel.

Purpose. These summary bits separate request location from the later direction and destination decisions. A request at the current floor is handled before dispatching another trip.

21 Requests Below and Direction Reversal

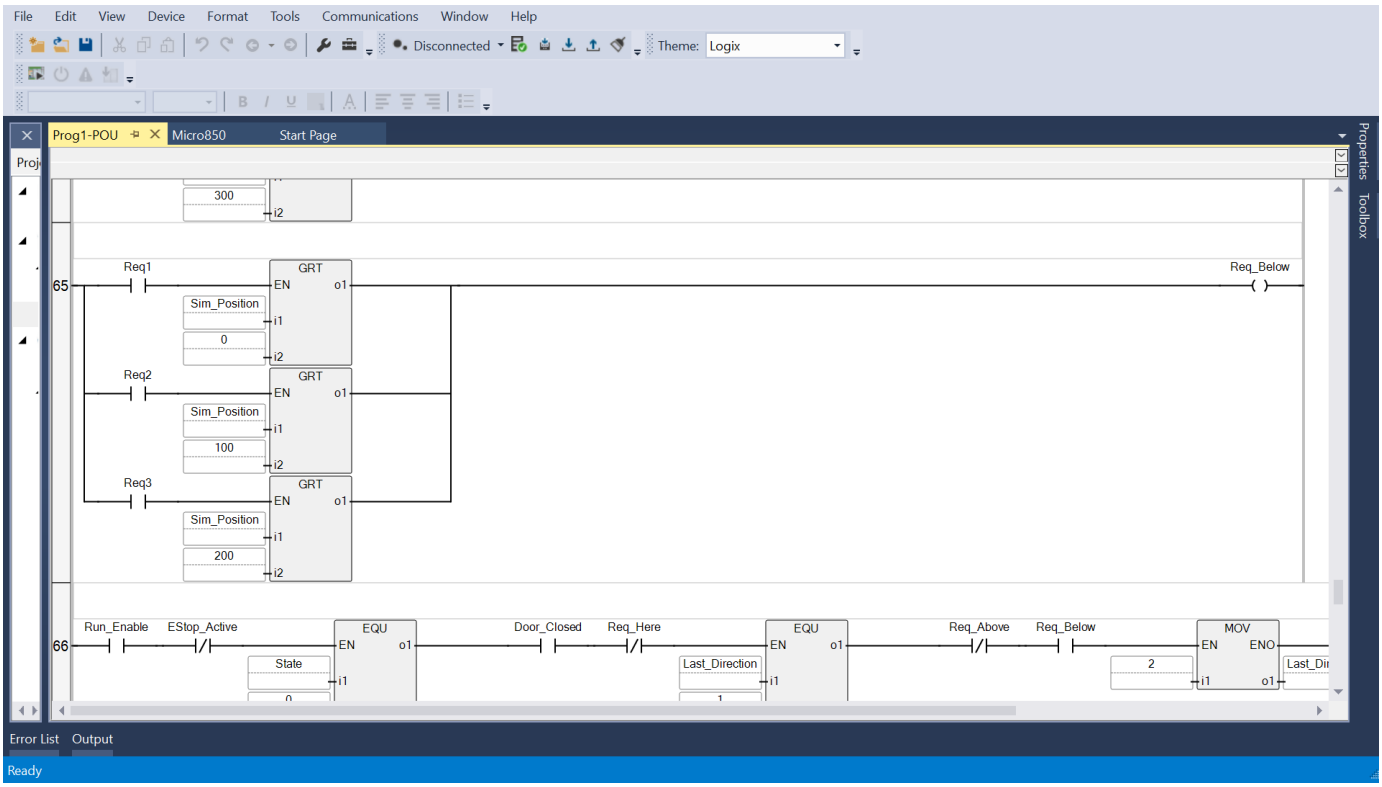


Figure 19. Ladder logic for requests below and direction reversal.

Rung 65. Req_Below is TRUE for Req1 with position above 0, Req2 with position above 100, or Req3 with position above 200.

Rung 66. With operation enabled, no E-stop, idle state, closed door, no request here, Last_Direction = 1, no request above, and a request below, MOV selects Last_Direction = 2.

Continuation. The rightmost destination label of the MOV is clipped in this capture and the next. The established direction tag is Last_Direction. The following page shows the complementary direction decision.

22 Direction Selection and Current Floor Service

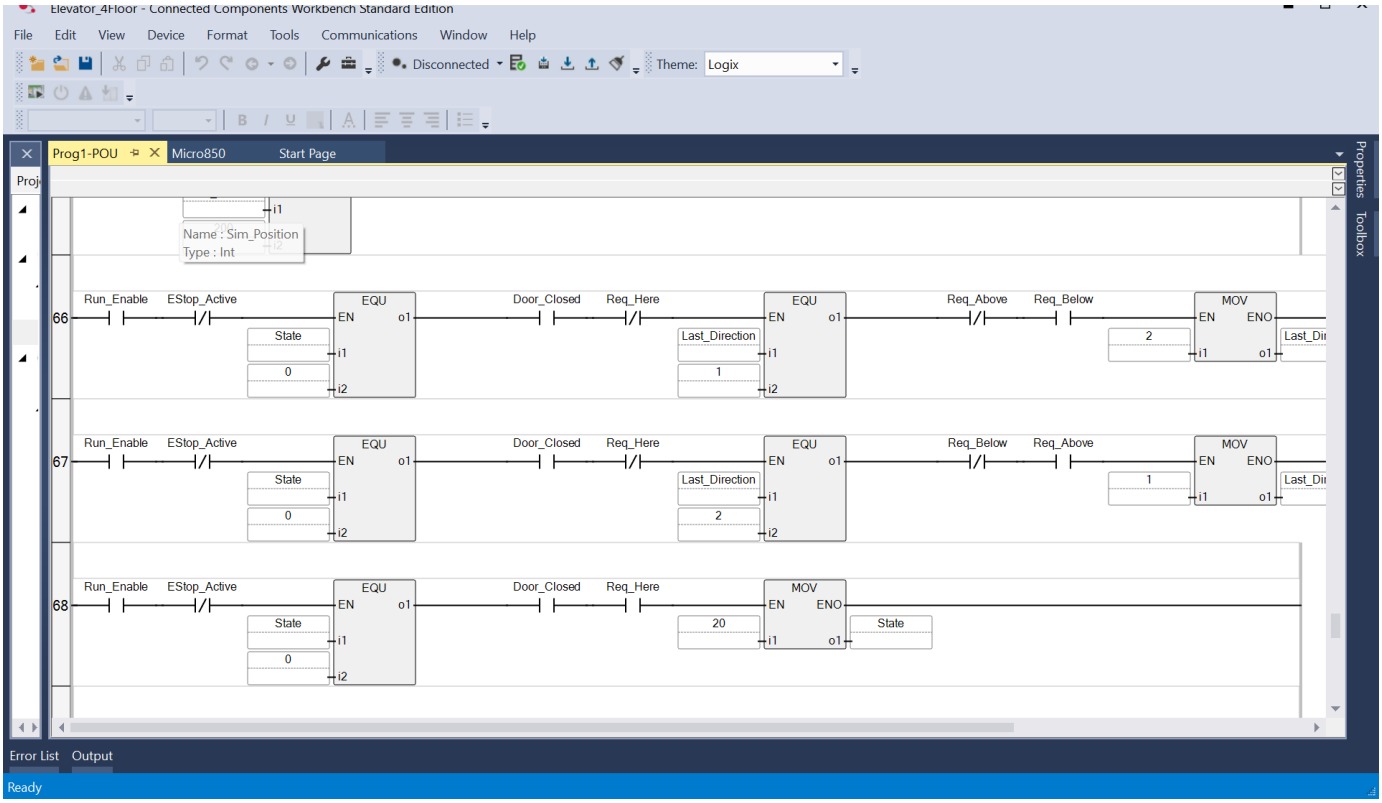


Figure 20. Ladder logic for direction selection and current floor service.

Rung 66. The retained upward direction changes to downward when only requests below remain and the idle service permissives are satisfied.

Rung 67. With Run_Enable TRUE, EStop_Active FALSE, State = 0, Door_Closed TRUE, Req_Here FALSE, Last_Direction = 2, Req_Below FALSE, and Req_Above TRUE, MOV selects Last_Direction = 1.

Rung 68. Run_Enable TRUE, no E-stop, idle state, Door_Closed TRUE, and Req_Here TRUE move 20 to State. This opens the door for a request at the current floor without initiating travel.

23 Target Initialization and Destination Selection

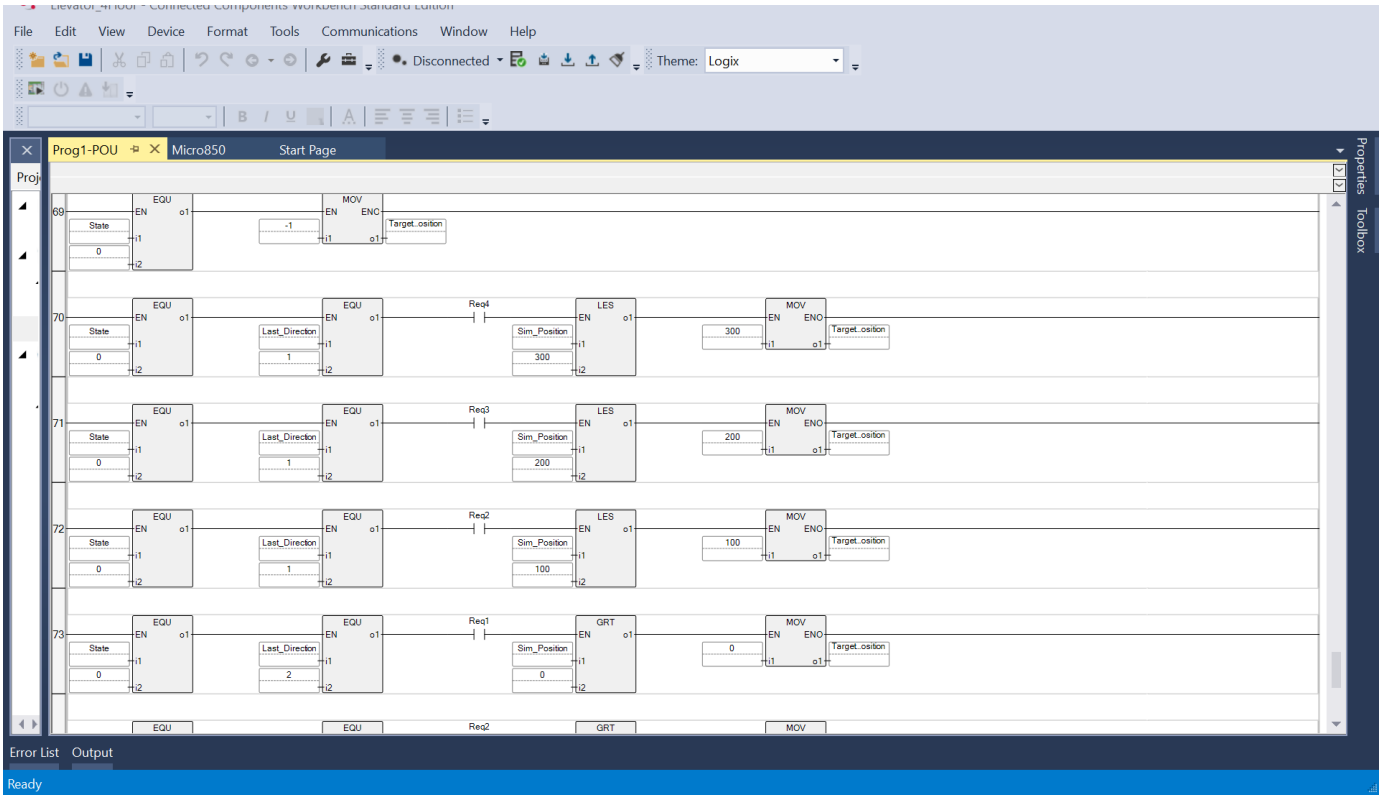


Figure 21. Ladder logic for target initialization and destination selection.

Rung 69. State = 0 moves -1 to Target_Position before checking destination candidates.

Rungs 70 to 72. At idle with Last_Direction = 1, requests above are checked in the order floor 4, floor 3, floor 2. Eligible branches write 300, 200, and 100 respectively. Later writes overwrite earlier ones so the nearest pending floor above is retained.

Rung 73. At idle with Last_Direction = 2, Req1 and a position above 0 write target 0. Floors 2 and 3 are checked afterward on the following page.

Locked target. These selection rungs execute only in state 0. Target_Position therefore remains unchanged during an active trip.

24 Downward Selection and Travel Dispatch

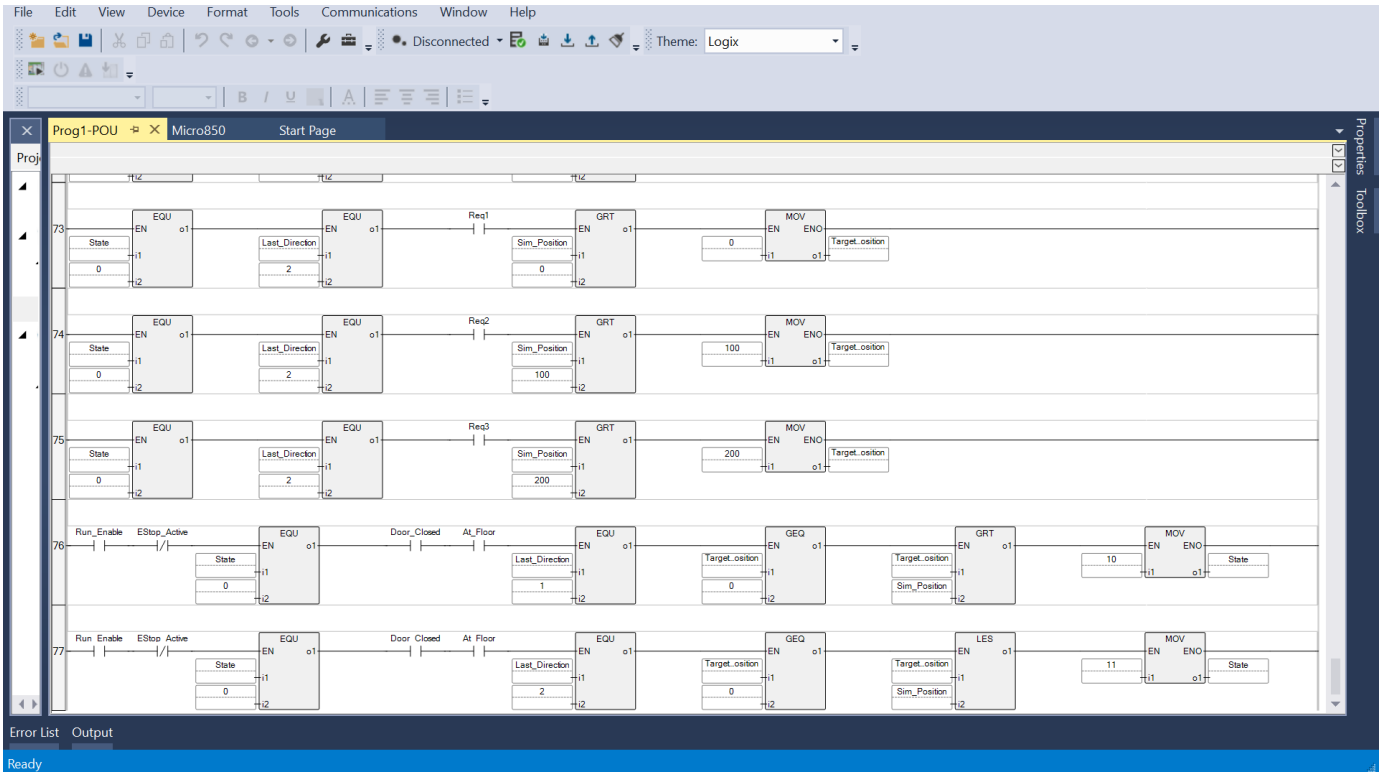


Figure 22. Ladder logic for downward selection and travel dispatch.

Rungs 73 to 75. At idle with Last_Direction = 2, eligible requests for floors 1, 2, and 3 write targets 0, 100, and 200 in that order. The last eligible write selects the nearest requested floor below.

Rung 76. Run_Enable TRUE, EStop_Active FALSE, State = 0, Door_Closed and At_Floor TRUE, Last_Direction = 1, Target_Position at least 0, and target greater than position move 10 to State.

Rung 77. The same dispatch permissives with Last_Direction = 2 and target below position move 11 to State.

Final capture. This is the replacement screenshot selected for the end of the program. It includes both complete travel-dispatch rungs.

25 Program Behavior and Engineering Conclusions

The program combines request memory with state-based control. Direction is retained while eligible requests remain ahead, and destination selection chooses the nearest pending floor in that direction. The target is locked during travel. A new call can be stored during the trip and handled after the current target is served.

Fault handling is placed above the normal sequence. A nonzero code latches the fault, forces state 99, clears stored requests, and removes operating permission. The first recorded code remains available for diagnosis until an accepted reset. Codes 1, 2, and 3 identify E-stop, invalid position, and travel timeout respectively.

Reset permission depends on the current conditions rather than the Reset button alone. A valid floor, closed door, released Start and Stop, cleared fault causes, and released test fixtures are required. An accepted reset does not enable motion. A new Start edge is required afterward.

The 23 functional checks documented in the separate project report covered normal travel, door sequencing, stored and additional requests, Stop and restart behavior, injected faults, blocked reset, and recovery. The watchdog correction was verified through normal operation and a final 60-second blocked-travel test. The travel E-stop test stopped the simulation at position 248 with outputs disabled and requests cleared.

I used this project to practice ladder scan order, rising-edge detection, latching, timer-based supervision, and systematic troubleshooting. The completed scope is a Micro850 simulation. Physical I/O commissioning, a completed HMI, and a dedicated hardware safety system are outside this implementation.

Documentation coverage: rungs 1 through 77 across 22 selected screenshots. Some images contain partial or repeated adjacent rungs because of the editor viewport. The explanations follow the complete program sequence, and no missing image area has been reconstructed or altered.